



**Security
OpenSSL**

Creation and Management of Certificates

Roberta Daidone

roberta.daidone@iet.unipi.it

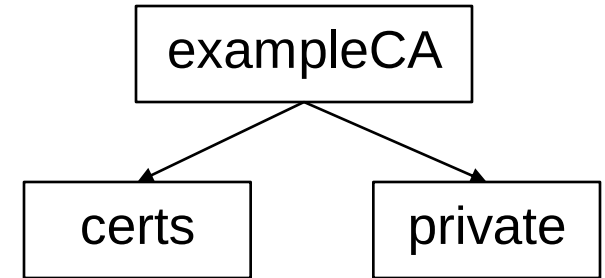
What are we going to do?

- Setup of a Certification Authority
- Creation of a self-signed *root certificate*
- Creation of a certificate request
- Creation of a X509 certificate
- Sign and verify a file hashing
- Revoke a certificate and issue a CRL

Creating the CA's environment

- Create the directories for the CA

```
$ mkdir exampleCA  
$ cd exampleCA  
$ mkdir certs private
```



- Change permissions for ***private*** directory:

```
$ sudo chmod 700 private
```

- To verify:

```
$ ls -l
```

Creating the CA's environment

- The ***serial*** file keeps track of certificates' serial. We'll initialize it to contain the number 0x01.

```
$ echo '01'> serial
```

- The ***index.txt*** file is a database of sorts that keeps track of the certificates that have been issued by the CA.

```
$ touch index.txt
```

- Create the ***openssl.cnf*** configuration file.

```
$ touch openssl.cnf
```

Configure OPENSSL_CONF

- Set the `OPENSSL_CONF` environment variable to tell the OpenSSL where to find the ***openssl.cnf*** file

```
$ OPENSSL_CONF=./openssl.cnf  
$ export OPENSSL_CONF
```

- To verify:

```
$ echo $OPENSSL_CONF
```

... and you should see the `openssl.cnf` path

How is openssl.cnf structured?

- The OpenSSL configuration file is organized in sections. Each section has a case-sensitive name.

[name]

- Each section contains a set of keys with an associated value. Also keys case-sensitive.

keyword = value

- two important sections:
 - `[ca]` to setup the **ca** command. It allows you to issue, verify and sign certificates, or CRLs.
 - `[req]` to setup the **req** command. It allows you to issue the self-signed certificate, or ask for a certificate.

openssl.cnf

```
[ ca ]
default_ca          = exampleCA

[ exampleCA ]
dir                  = .           #exampleCA dir
certificate          = $dir/cacert.pem
database             = $dir/index.txt
serial              = $dir/serial
new_certs_dir        = $dir/certs
private_key          = $dir/private/privkey.pem

default_days         = 365
default_crl_days     = 7
default_md           = md5
```

openssl.cnf

```
policy = exampleca_policy
x509_extensions = certificate_extensions

[ exampleca_policy ]
commonName = supplied
stateOrProvinceName = supplied
countryName = supplied
organizationName = supplied
organizationalUnitName = optional

[ certificate_extensions ]
basicConstraints = CA:false
```


Creating a self-signed root certificate

- You need some configuration file additions for generating a self-signed root certificate. The **req** command has the following section in openssl.cnf

```
[ req ]
default_bits          = 2048
default_keyfile        = ./private/privkey.pem
default_md             = md5
prompt                = no
distinguished_name     = root_ca_distinguished_name
x509_extensions        = root_ca_extensions
```

Creating a self-signed root certificate

```
[ root_ca_distinguished_name ]
  commonName                = Daidone CA
  stateOrProvinceName       = Italy
  countryName                = EU
  emailAddress               = daidone@mycert.it
  organizationName           = Root Cert Authority

[ root_ca_extensions ]
  basicConstraints           = CA:true
```

- Now you can issue your self-signed **root certificate** the CA's **private** and **public keys**

Creating a self-signed root certificate

- Execute the **req** command

```
$ cd exampleCA/  
$ openssl req -x509 -newkey rsa:2048 -out  
cacert.pem -outform PEM
```

- `-x509` is the certificate format
- `-newkey rsa:2048` a pair of RSA keys will be generated, of 2048 bits each
- `-out <file>` certificate output file
- `-outform` output format

Effects

- You are prompted for a passphrase to encrypt your private key.
- Two files are generated:
 - ***privkey.pem*** in *exampleCA/private*.
 - ***cacert.pem*** in *exampleCA/*.
- To visualize the certificate we use the **x509** command:

```
$ openssl x509 -in cacert.pem -text -noout
```

- `-text` prints out the certificate in text form.
- `-noout` prevents output of the encoded version of the request

Issuing a certificate request

- As part of the process to generate a certificate request, a new key pair is also generated.
- Start with a clean shell without the OPENSSL_CONF environment variable set, so that the default configuration file is used.
- Create the user directory (just to distinguish)

```
$ mkdir exampleUSER  
$ cd exampleUSER
```

- Generate a certificate request

```
$ openssl req -newkey rsa:1024 -keyout testkey.pem  
-keyform PEM -out testreq.pem
```

Effects

- OPENSSL_CONF is not set => you are prompted for more information
- The first passphrase that is used to encrypt the private key.
- The challenge phrase is stored in the certificate request ...and is otherwise ignored by OpenSSL!
- As a result two files are created:
 - ***testkey.pem*** for the private key
 - ***testreq.pem*** for the certificate request
- To visualize the request:

```
$ openssl req -in testreq.pem -text -noout
```

Issuing a certificate from a request

- Make sure you are in the shell with the OPENSSL_CONF variable set.

- Issue the command to generate the certificate:

```
$ openssl ca -in testreq.pem
```

- Effects:
 - OpenSSL asks for the passphrase associated to the CA's private key.
 - After displaying the subject's distinguished name, OpenSSL prompts you for confirmation to sign the certificate.
 - The certificate file (**<serial>.pem**) is in the **exampleCA/certs** directory.

Create and sign a digest

- Create the ***data.txt*** file

```
$ echo 'Please, sign me' > data.txt
```

- Create the digest file ***hash***:

```
$ openssl dgst -sha1 < data.txt > hash
```

- Sign the ***hash*** file to the ***signature*** file:

```
$ openssl rsautl -sign -inkey testkey.pem -keyform  
PEM -in hash > signature
```


Public key retrieval

- By means of the `rsa` command you obtain the public key from the ***testkey.pem*** file

```
$ openssl rsa -in testkey.pem -out public.pem  
-outform PEM -pubout
```

- `-pubout` by default a private key is output: with this option a public key will be output instead.
- To visualize public key:

```
$ openssl rsa -in public.pem -text -noout -pubin
```

 - `-pubin` by default a private key is read from the input file: with this option a public key is read instead.

Signature verification

- Verify ***signature*** file to obtain ***verified*** file containing the digest.

```
$ openssl rsautl -verify -inkey public.pem -keyform  
PEM -pubin -in signature > verified
```

- Verify that ***verified*** and ***hash*** are identical:

```
$ diff -s verified hash
```

- s reports when two files are the same

Certificate revocation

- To revoke a certificate you need a copy of a certificate you want to revoke.

```
$ cp cert/01.pem testrevoke.pem
```

- Use the `ca` command with the `-revoke` option, specifying the name of the copy of the certificate we created.

```
$ openssl ca -revoke testrevoke.pem
```

- Effects:
 - OpenSSL prompts us for the passphrase protecting the CA's private key.
 - In the `index.txt` file, an `R` appears in the line of our certificate.

CRL

- To issue a CRL, use the `ca` command with the `-gencrl` option.

```
$ openssl ca -gencrl -out CRLfile.pem -keyform PEM
```

- Effects:
 - OpenSSL prompts us for the passphrase protecting the CA's private key.
 - If the command completes without writing anything to stdout indicates success and the ***CRLfile.pem*** is generated

- To visualize the CRL:

```
$ openssl crl -in CRLfile.pem -text -noout
```

Exercise

- Create a key pair using the OpenSSL command line tool
- Substitute keys you used for the RSA C exercise with these you have already generated
- Readapt the code of client and server to use new keys

Exercise

- Create a key pair using the OpenSSL command line tool
- Substitute keys you used for the RSA C exercise with these you have already generated
- Readapt the code of client and server to use new keys
 - Suggestion: keys you used last time were RSA keys, these are EVP_PKEY keys.